



UNIDADE III

Modelagem de Banco de Dados e NoSQL

Profa. Dra. Vanessa Lessa

Otimização de Consultas em SQL Server

- Em ambientes de alto desempenho, a forma como as consultas são estruturadas e os dados organizados impacta diretamente o tempo de resposta das aplicações. À medida que o volume de dados cresce, adotar estratégias eficientes torna-se indispensável para garantir escalabilidade e reduzir gargalos.

Por que a Otimização é Essencial?

Volume de Dados

- Consultas sem índices adequados tornam-se progressivamente mais custosas conforme os dados crescem.

Acessos Simultâneos

- Múltiplos usuários acessando o banco ao mesmo tempo exigem estratégias que evitem gargalos severos.

Escalabilidade

- Planejar a indexação de forma proativa garante desempenho sustentável mesmo com crescimento futuro.
- O uso adequado de índices e ajustes de performance assume papel central na otimização do SQL Server, permitindo identificar operações custosas e oportunidades de melhoria contínua.

O Papel dos Índices no SQL Server

- Índices são estruturas auxiliares que permitem ao SQL Server localizar informações de forma eficiente, **reduzindo o volume de dados examinados** durante a execução de uma consulta.
- Sem índices adequados, o sistema realiza varreduras completas — linha por linha —, tornando as operações progressivamente mais custosas.
- A utilização de índices transforma o processamento de consultas: em vez de acessar dados brutos diretamente, o sistema consulta uma estrutura organizada e otimizada para buscas, reduzindo drasticamente o número de operações de leitura.

Impacto dos Índices na Performance

Sem Índices

- Varredura completa da tabela.
- Alto consumo de recursos.
- Gargalos em acessos simultâneos.
- Tempo de resposta elevado.

Com Índices Adequados

- Busca direcionada e eficiente.
 - Menor consumo de I/O.
 - Estabilidade sob cargas elevadas.
 - Desempenho previsível e escalável.
-
- A aplicação criteriosa de índices contribui para a estabilidade e a previsibilidade do desempenho, mesmo em ambientes corporativos com grandes volumes de dados.

Dualidade dos Índices: Leitura vs. Escrita

Aceleração de Leituras

- Índices reduzem drasticamente o custo de consultas com filtros, junções e ordenações frequentes.

Custo em Escritas

- Inserções, atualizações e exclusões exigem manutenção dos índices, introduzindo sobrecarga adicional.

Abordagem Equilibrada

- A criação de índices deve considerar tanto o desempenho das consultas quanto o custo de manutenção das estruturas.

Observação:

- A criação indiscriminada de índices pode causar impacto negativo em operações de inserção, atualização e exclusão. É fundamental avaliar o perfil das consultas antes de definir uma estratégia de indexação.

Tipos de Índices: Índice Clustered

- No índice **clustered**, os dados são armazenados fisicamente na ordem definida pela chave do índice — a própria tabela é organizada conforme esse critério.

Isso o torna eficiente para:

- Consultas com intervalos de valores.
- Ordenações frequentes.
- Recuperação sequencial de dados.

- Como a tabela comporta **apenas um índice clustered**, sua escolha é uma decisão estratégica.
- Colunas com alta seletividade e uso frequente em filtros e junções são candidatas naturais.

Atenção na Escolha

- Uma escolha inadequada do índice clustered compromete o desempenho geral, pois afeta diretamente a forma como todos os dados são armazenados e acessados na tabela.

Índices Nonclustered e Compostos

Nonclustered

- Estruturas independentes da organização física da tabela.
- Uma tabela pode ter **múltiplos índices nonclustered**, cada um atendendo a diferentes padrões de consulta sem alterar a ordem física dos dados.

Cobertura de Consultas

- Um índice é **abrangente** quando contém todas as colunas necessárias para atender a uma consulta, eliminando leituras adicionais à tabela base.

Índices Compostos

- Formados por mais de uma coluna.
- A **ordem das colunas** influencia diretamente a eficácia — colunas mais seletivas devem ser posicionadas estrategicamente.

Cardinalidade e Estatísticas

Cardinalidade das Colunas

- A cardinalidade — quantidade de valores distintos — influencia diretamente o benefício da indexação:
- Alta cardinalidade: filtra registros com maior precisão, maximizando ganhos.
- Baixa cardinalidade: muitos registros compartilham valores, reduzindo a seletividade do índice.

Estatísticas Atualizadas

- As estatísticas informam ao otimizador como os dados estão distribuídos.
- Índices sem estatísticas atualizadas levam a estimativas incorretas e planos subótimos, comprometendo o desempenho mesmo quando a indexação parece adequada.

Planos de execução

- **Analisando Planos de Execução**
- O plano de execução descreve detalhadamente as etapas que o SQL Server seguirá para recuperar os dados solicitados: escolha de índices, tipo de acesso e ordem das operações.
- Interpretar corretamente essas informações permite **identificar gargalos e ajustar a estratégia de indexação** de forma fundamentada.
- **O que o Plano de Execução Revela**
 - **Varreduras vs. Buscas Direcionadas**

Varreduras completas frequentes indicam ausência de índices apropriados ou consultas formuladas de forma que inviabilizam seu uso.
 - **Custo Relativo das Operações**

O SQL Server atribui estimativas de custo a cada etapa, indicando quais consomem maior proporção dos recursos — candidatas naturais à revisão.
 - **Eficácia dos Índices Existentes**

Índices raramente utilizados ou ignorados pelo otimizador podem indicar redundância ou inadequação ao padrão de consultas do sistema.

Gargalos Comuns Identificados no Plano

Funções em Colunas

- Expressões com funções aplicadas diretamente às colunas podem impedir o uso de índices, mesmo quando presentes.

Conversões Implícitas

- Incompatibilidade de tipos de dados força conversões que inviabilizam a utilização eficiente dos índices disponíveis.

Junções sem Índice

- Chaves estrangeiras sem indexação resultam em custos elevados nas junções entre tabelas, visíveis no plano de execução.

Estatísticas Desatualizadas

- Levam o otimizador a subestimar ou superestimar custos, resultando em escolhas inadequadas de planos de execução.
- A análise contínua dos planos de execução permite adaptar a estratégia de indexação ao longo do tempo, evitando degradação gradual do desempenho.

Fragmentação: O Inimigo Silencioso

- Operações constantes de inserção, atualização e exclusão alteram a organização interna dos índices, gerando **fragmentação**:
as páginas de dados deixam de estar contíguas, aumentando o número de operações de leitura e o tempo de resposta das consultas.
- A fragmentação é particularmente comum em tabelas com grande volume de alterações. Seu impacto manifesta-se no aumento do tempo de resposta e no maior consumo de recursos de entrada e saída.

Reorganização vs. Reconstrução de Índices

Reorganização

- Indicada para **fragmentação moderada**.
- Reorganiza as páginas de forma mais eficiente sem recriar a estrutura.
- Menos invasiva, pode ser executada com maior frequência em sistemas com carga contínua.

Reconstrução

- Indicada para **fragmentação severa**.
- Recria completamente a estrutura, eliminando fragmentação e reorganizando os dados de forma otimizada.
- Mais custosa — deve ser planejada em janelas de manutenção.
- Nem toda fragmentação exige intervenção imediata.

Avaliar o grau de fragmentação e o padrão de uso do banco permite decidir quando e como agir, equilibrando custo e benefício.

Manutenção, Estatísticas e Automação

Índices + Estatísticas

- Operações de manutenção que reorganizam ou recriam índices, geralmente, atualizam as estatísticas associadas, contribuindo para decisões mais acertadas do otimizador.
- Índices e estatísticas devem ser tratados como **componentes interdependentes**.

Automação Recomendada

- Em ambientes corporativos, definir procedimentos automáticos para:
- Avaliar nível de fragmentação.
- Reorganizar ou reconstruir índices.
- Atualizar estatísticas periodicamente.
- Isso reduz dependência de intervenções manuais e aumenta a consistência das ações realizadas.

Interatividade

Analise as afirmações:

- I. A análise dos planos de execução permite identificar gargalos de desempenho, como varreduras completas de tabelas e índices ignorados pelo otimizador, subsidiando ajustes na estratégia de indexação.
- II. A fragmentação de índices ocorre exclusivamente em tabelas estáticas e não sofre influência de operações de inserção, atualização ou exclusão de dados.
- III. Estatísticas atualizadas são fundamentais para que o otimizador de consultas estime corretamente os custos das operações e selecione planos de execução eficientes.
- IV. A criação excessiva de índices tende a melhorar simultaneamente o desempenho de leitura e escrita, sendo recomendada em sistemas com alta taxa de transações.

Está correto o que se afirma em:

- a) I e IV, apenas.
- b) I e III, apenas.
- c) II e III, apenas.
- d) II e IV, apenas.
- e) III e IV, apenas.

Resposta

Analise as afirmações:

- I. A análise dos planos de execução permite identificar gargalos de desempenho, como varreduras completas de tabelas e índices ignorados pelo otimizador, subsidiando ajustes na estratégia de indexação.
- II. A fragmentação de índices ocorre exclusivamente em tabelas estáticas e não sofre influência de operações de inserção, atualização ou exclusão de dados.
- III. Estatísticas atualizadas são fundamentais para que o otimizador de consultas estime corretamente os custos das operações e selecione planos de execução eficientes.
- IV. A criação excessiva de índices tende a melhorar simultaneamente o desempenho de leitura e escrita, sendo recomendada em sistemas com alta taxa de transações.

Está correto o que se afirma em:

- a) I e IV, apenas.
- b) I e III, apenas.
- c) II e III, apenas.
- d) II e IV, apenas.
- e) III e IV, apenas.

Performance: Por que Monitorar Queries?

- O monitoramento de queries é o ponto de partida de qualquer estratégia consistente de ajuste de performance. Antes de propor otimizações, é fundamental compreender **como o banco de dados está sendo utilizado**, quais consultas consomem mais recursos e onde ocorrem os principais gargalos.
- Essa observação sistemática substitui decisões baseadas em suposições por intervenções fundamentadas em dados reais, aumentando significativamente a eficácia das ações de otimização.

Performance é um Processo Contínuo

Uso Dinâmico

- O padrão de uso, raramente, é estático. Novas funcionalidades, crescimento de dados e mudanças no perfil dos usuários transformam consultas adequadas em gargalos críticos.

Ação Proativa

- O monitoramento proativo identifica tendências de degradação antes que afetem a experiência do usuário, evitando intervenções reativas e arriscadas.

Visão Integrada

- O desempenho depende da modelagem dos dados, da carga de trabalho, do comportamento das aplicações e da capacidade do hardware — não de um fator isolado.
- Ajustes realizados apenas em situações de crise são mais complexos e arriscados do que intervenções preventivas baseadas em monitoramento contínuo.

Ferramentas e Métricas de Monitoramento

- As ferramentas disponíveis no SQL Server transformam o desempenho em um fenômeno **observável e mensurável**, permitindo que decisões de otimização sejam tomadas com base em evidências concretas — e não em percepções subjetivas de lentidão.
- **Principais Métricas de Performance**
 - **Tempo de Execução**

Tempo total de resposta, tempo de CPU e tempo de espera por recursos — perspectivas complementares sobre o comportamento da consulta.
 - **Consumo de CPU**

Consultas intensivas em processamento podem monopolizar recursos e degradar outras operações em ambientes com múltiplas aplicações concorrentes.
 - **Leitura e Escrita**

Alto volume de leituras pode indicar ausência de índices; muitas gravações podem sinalizar processos de carga mal otimizados.
 - **Esperas (Waits)**

Esperas por bloqueios, I/O ou recursos internos revelam pontos de contenção — concorrência, configuração inadequada ou limitações de infraestrutura.

Frequência, Variabilidade e Histórico

Consultas Frequentes

- Queries aparentemente simples, executadas milhares de vezes, podem exercer impacto significativo sobre o desempenho global.

Variabilidade de Desempenho

- Tempos de resposta inconsistentes entre execuções indicam planos de execução instáveis ou mudanças na distribuição dos dados.

Análise Histórica

- A evolução das métricas ao longo do tempo permite identificar tendências de degradação e planejar ajustes proativos.

Métricas como Base para Priorização

- A interpretação correta das métricas exige conhecimento do contexto operacional — valores elevados em um ambiente podem ser aceitáveis em outro, dependendo do volume de dados e das expectativas de desempenho.

Priorização por Impacto

- Classificar consultas por frequência, duração e criticidade permite concentrar esforços onde o ganho positivo será maior.

Comunicação entre Equipes

- Dados objetivos facilitam o diálogo entre administradores, desenvolvedores e gestores, tornando decisões mais colaborativas e fundamentadas.

Cultura de Observabilidade

- O uso contínuo das ferramentas fortalece a postura proativa da equipe, identificando oportunidades de melhoria de forma sistemática.

Análise de Queries Problemáticas

- Identificar uma consulta lenta é apenas o primeiro passo. Compreender **por que** ela apresenta desempenho insatisfatório exige examinar sua estrutura, filtros, junções e a forma como interage com as estruturas internas do banco de dados.
- **Fontes Comuns de Gargalos em Queries**
 - **Filtros Ineficientes**

Consultas com filtros pouco seletivos retornam mais dados que o necessário, sobrecarregando CPU e memória.
 - **Junções Inadequadas**

Junções mal definidas em tabelas volumosas geram explosão de combinações intermediárias e elevam o custo computacional.
 - **Subconsultas Excessivas**

Uso inadequado de subconsultas pode resultar em múltiplas execuções implícitas; reescrevê-las como junções gera planos mais eficientes.
 - **Funções em Colunas Filtradas**

Funções aplicadas diretamente a colunas de filtro impedem o uso de índices, forçando varreduras completas de tabela.

Varreduras Completas e Volume de Dados

Varreduras de Tabela (Full Scans)

- Operações que exigem varreduras completas de grandes tabelas indicam ausência de filtros eficientes ou de estruturas de apoio adequadas. A simples reformulação da condição de busca pode permitir o uso mais eficiente de índices existentes, reduzindo drasticamente o custo da operação.

Volume Retornado

- Avaliar se todos os dados retornados são realmente necessários e limitar o escopo da consulta reduz o impacto sobre o sistema e melhora a experiência do usuário — mesmo em queries bem estruturadas.

Contexto, Validação e Aprendizado Organizacional

Contexto de Execução

- Queries executadas em horários de pico ou junto a operações intensivas podem degradar por concorrência de recursos — o sistema como um todo deve ser considerado.

Ciclo Iterativo de Validação

- Após ajustar uma query, monitorar novamente seu comportamento confirma se os ganhos foram alcançados e se não houve impacto negativo em outras operações.

Boas Práticas de Desenvolvimento

- Identificar padrões recorrentes de ineficiência permite estabelecer diretrizes para escrita de consultas, reduzindo a incidência de problemas futuros e fortalecendo a qualidade do código SQL.

Otimização Estrutural do Banco de Dados

- Enquanto a reescrita de consultas gera ganhos imediatos, a revisão das **estruturas internas** do banco de dados sustenta esses ganhos ao longo do tempo e acomoda o crescimento do volume de dados e da complexidade do sistema.

Índices: Criação, Revisão e Manutenção

Criação de Índices

- Índices bem definidos reduzem o custo das consultas. Sua eficácia depende da adequação às colunas usadas em filtros, junções e ordenações.

Remoção de Redundâncias

- Índices redundantes ou pouco utilizados consomem armazenamento e aumentam o custo de operações de escrita sem oferecer benefícios proporcionais.

Manutenção Regular

- Operações de modificação geram fragmentação ao longo do tempo. Rotinas de reorganização ou reconstrução preservam o desempenho e a previsibilidade do tempo de resposta.

Estatísticas, Armazenamento e Configurações

Estatísticas do Banco de Dados

- As estatísticas fornecem ao otimizador informações sobre a distribuição dos dados e influenciam diretamente as estimativas de custo. **Estatísticas desatualizadas** levam à escolha de planos subótimos, mesmo com índices corretos. A atualização periódica é prática fundamental.

Armazenamento e Configurações

- A distribuição de tabelas e índices em discos impacta leitura e escrita. Parâmetros de memória, paralelismo e comportamento do otimizador também influenciam a execução das queries — ajustes devem ser feitos com cautela, pois afetam o sistema globalmente.

Trade-offs, Escalabilidade e Documentação

Desempenho vs. Confiabilidade

- Alterações que maximizam performance podem aumentar a complexidade ou reduzir a margem de segurança. Avaliar trade-offs é essencial para não comprometer integridade dos dados.

Escalabilidade Proativa

- Estruturas adequadas para volumes menores podem se tornar inadequadas com o crescimento. Antecipar esse crescimento reduz a probabilidade de gargalos futuros.

Documentação das Alterações

- Registrar decisões, motivos e resultados observados fortalece a governança, a rastreabilidade e apoia futuras intervenções na estrutura do banco de dados.

Integração entre Monitoramento e Otimização

- O monitoramento fornece a **visibilidade** necessária para identificar problemas; a otimização transforma essa visibilidade em **ações concretas** de melhoria. Intervenções isoladas oferecem ganhos limitados — a combinação de ajustes em diferentes níveis produz resultados mais consistentes e duradouros.

Queries + Estrutura

- Índices bem definidos e estatísticas atualizadas potencializam os ganhos obtidos com a reescrita das consultas.

Código + Configuração

- A qualidade do SQL e as configurações do ambiente são igualmente determinantes para o desempenho final do sistema.

Equipes Integradas

- Administradores, desenvolvedores e arquitetos precisam atuar de forma coordenada, com decisões baseadas em métricas e evidências compartilhadas.

Interatividade

Analise as afirmações:

- I. O monitoramento contínuo de queries permite identificar tendências de degradação de desempenho e fundamentar ajustes proativos antes que o impacto seja percebido pelos usuários finais.
- II. Métricas de tempo de execução, consumo de CPU, leitura/escrita e esperas devem ser interpretadas isoladamente, pois a correlação entre elas não influencia o diagnóstico de gargalos.
- III. A análise de queries problemáticas envolve compreender a estrutura das consultas, seus filtros, junções e o contexto de execução, buscando corrigir causas reais e não apenas sintomas aparentes.
- IV. A documentação das alterações realizadas durante ajustes de performance contribui para a governança do banco de dados, permitindo rastreabilidade, aprendizado organizacional e decisões futuras mais consistentes.

Interatividade

Está correto o que se afirma em:

- a) I e II, apenas.
- b) II e III, apenas.
- c) I e IV, apenas.
- d) II e IV, apenas.
- e) I e III, apenas.

Resposta

Analise as afirmações:

- I. O monitoramento contínuo de queries permite identificar tendências de degradação de desempenho e fundamentar ajustes proativos antes que o impacto seja percebido pelos usuários finais.
- II. Métricas de tempo de execução, consumo de CPU, leitura/escrita e esperas devem ser interpretadas isoladamente, pois a correlação entre elas não influencia o diagnóstico de gargalos.
- III. A análise de queries problemáticas envolve compreender a estrutura das consultas, seus filtros, junções e o contexto de execução, buscando corrigir causas reais e não apenas sintomas aparentes.
- IV. A documentação das alterações realizadas durante ajustes de performance contribui para a governança do banco de dados, permitindo rastreabilidade, aprendizado organizacional e decisões futuras mais consistentes.

Está correto o que se afirma em:

e) I e III, apenas.

Da Teoria à Prática

- Após compreender os fundamentos de modelagem, gerenciamento e otimização, torna-se essencial aplicar esse conhecimento na construção de soluções completas que reflitam situações reais.

Impacto das Decisões Técnicas

- A abordagem prática permite compreender como decisões de projeto, definição de estruturas e escolhas técnicas impactam diretamente o **funcionamento**, o **desempenho** e a **manutenção** de um banco de dados em cenários concretos.

Desenvolvimento de uma Solução Relacional Completa

- Este tópico apresenta o desenvolvimento de uma solução relacional completa no SQL Server, integrando conceitos de modelagem, definição de tabelas e relacionamentos, além da implementação de consultas e operações avançadas.

Modelagem

- Projeto conceitual, lógico e físico do banco de dados.

Implementação

- Criação de tabelas, chaves e relacionamentos no SQL Server.

Consultas Avançadas

- Exploração do potencial do SQL Server para resolver problemas reais.

Sistema de Pedido de Vendas

- O caso prático consiste no desenvolvimento de um **sistema de pedido de vendas** utilizado por uma empresa comercial para registrar clientes, controlar pedidos, gerenciar produtos e acompanhar pagamentos.
- Esse tipo de sistema é comum em ambientes de **comércio eletrônico**, lojas físicas informatizadas e plataformas corporativas de gestão empresarial, sendo essencial para garantir organização das informações, rastreabilidade das transações e apoio à tomada de decisões gerenciais.

Entidades do Sistema

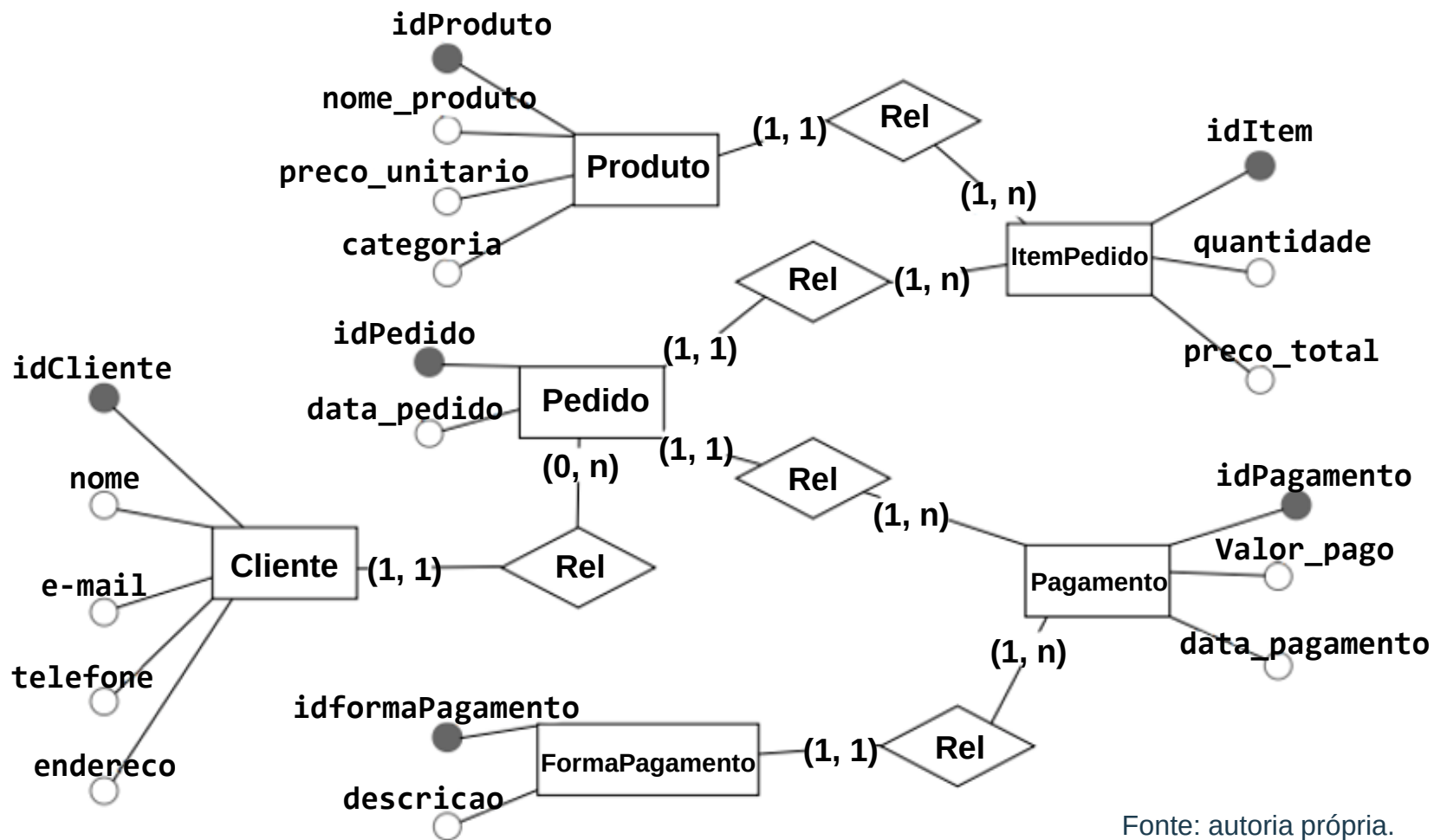
- **Cliente**
id_cliente (PK), nome, e-mail, telefone, endereço
- **Pedido**
id_pedido (PK), data_pedido, id_cliente (FK)
- **Produto**
id_produto (PK), nome_produto, preco_unitario, categoria
- **ItemPedido**
id_item (PK), id_pedido (FK), id_produto (FK), quantidade, preco_total
- **FormaPagamento**
id_forma (PK), descrição
- **Pagamento**
id_pagamento (PK), id_pedido (FK), id_forma (FK), valor_pago, data_pagamento

Relacionamentos entre Entidades

- **Cliente → Pedido (1:N)**
Um cliente pode realizar vários pedidos; cada pedido pertence a um único cliente.
- **Pedido → ItemPedido (1:N)**
Um pedido é composto por vários itens; cada item pertence a um único pedido.
- **Produto → ItemPedido (1:N)**
Um produto pode aparecer em vários itens de pedidos diferentes.
- **Pedido → Pagamento (1:1 ou 1:N)**
Um pedido pode ter um ou mais pagamentos, dependendo da regra de negócio.
- **FormaPagamento → Pagamento (N:1)**
Cada pagamento usa uma única forma; uma forma pode ser usada em vários pagamentos.
- A entidade **ItemPedido** resolve a relação muitos-para-muitos entre Pedido e Produto, funcionando como entidade associativa.
- **Observação Importante:** Ao analisar casos práticos, o estudante deve focar não apenas no resultado final, mas também nas **decisões tomadas durante a modelagem e a implementação**, compreendendo os motivos que justificam cada escolha.

Modelo Conceitual: Diagrama Entidade-Relacionamento (DER)

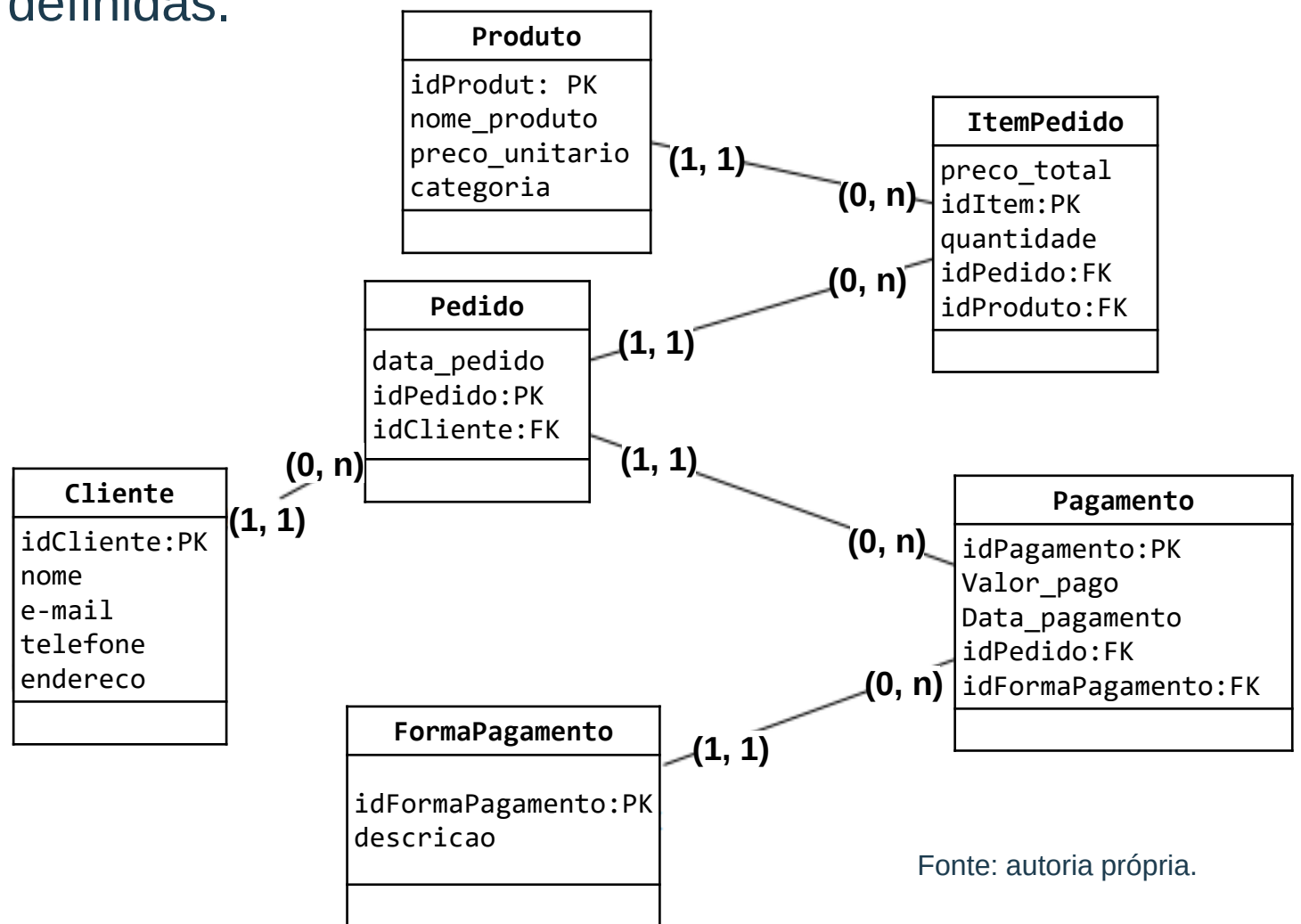
- No modelo conceitual, representamos as entidades, os relacionamentos com as cardinalidades adequadas e os atributos inseridos em cada entidade. A ferramenta **brModelo** é amplamente utilizada para essa etapa.



Fonte: autoria própria.

Modelo Lógico: Transição para o Modelo Lógico

- Na transição para o modelo lógico, os relacionamentos são convertidos em estruturas de tabelas relacionais com **chaves primárias (PK)** e **chaves estrangeiras (FK)** explicitamente definidas.



Fonte: autoria própria.

Modelo Físico: Script SQL — Versão Inicial (brModelo)

O brModelo permite gerar automaticamente o script para o modelo físico. A versão inicial gerada pela ferramenta serve como ponto de partida:

```
CREATE TABLE Cliente (  
    idCliente INT PRIMARY KEY,  
    nome INT,  
    e-mail INT,  
    telefone INT,  
    endereco INT  
);
```

```
CREATE TABLE Pedido (  
    idPedido INT PRIMARY KEY,  
    data_pedido INT,  
    idCliente INT  
);
```

```
CREATE TABLE Produto (  
    idProduto INT PRIMARY KEY,  
    nome_produto INT,  
    preco_unitario INT,  
    categoria INT  
);
```

```
ALTER TABLE Pedido  
    ADD FOREIGN KEY(idCliente)  
    REFERENCES Cliente(idCliente);
```

- Como os tipos de dados e restrições não foram definidos nas etapas anteriores, o script gerado deve ser **aprimorado antes de executar no SGBD**.

Script SQL — Versão Aprimorada

Com tipos de dados e restrições adequados, o script final garante integridade e consistência dos dados:

```
CREATE TABLE Cliente (  
    id_cliente INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    telefone VARCHAR(20),  
    endereco VARCHAR(200)  
);  
  
CREATE TABLE Produto (  
    id_produto INT PRIMARY KEY,  
    nome_produto VARCHAR(100) NOT NULL,  
    preco_unitario DECIMAL(10,2) NOT  
NULL CHECK (preco_unitario >= 0),  
    categoria VARCHAR(50)  
);
```

```
CREATE TABLE Pedido (  
    id_pedido INT PRIMARY KEY,  
    data_pedido DATE NOT NULL DEFAULT  
CURRENT_TIMESTAMP,  
    id_cliente INT NOT NULL,  
    FOREIGN KEY (id_cliente)  
        REFERENCES Cliente(id_cliente)  
        ON DELETE CASCADE  
);
```

Script SQL: ItemPedido, FormaPagamento e Pagamento

```
CREATE TABLE ItemPedido (  
    id_item INT PRIMARY KEY,  
    id_pedido INT NOT NULL,  
    id_produto INT NOT NULL,  
    quantidade INT NOT NULL CHECK (quantidade > 0),  
    subtotal DECIMAL(10,2) NOT NULL CHECK (subtotal >= 0),  
    FOREIGN KEY (id_pedido)  
        REFERENCES Pedido(id_pedido) ON DELETE CASCADE,  
    FOREIGN KEY (id_produto)  
        REFERENCES Produto(id_produto)  
);
```

```
CREATE TABLE FormaPagamento (  
    id_forma INT PRIMARY KEY,  
    descricao VARCHAR(50) NOT NULL UNIQUE  
);
```

Script SQL: ItemPedido, FormaPagamento e Pagamento

```
CREATE TABLE Pagamento (  
    id_pagamento INT PRIMARY KEY,  
    id_pedido INT NOT NULL,  
    id_forma INT NOT NULL,  
    valor_pago DECIMAL(10,2) NOT NULL CHECK (valor_pago >= 0),  
    data_pagamento DATE NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (id_pedido) REFERENCES Pedido(id_pedido),  
    FOREIGN KEY (id_forma) REFERENCES FormaPagamento(id_forma)
```

Observação sobre a Estrutura do Banco

Decisões de Modelagem

- A estrutura do banco adotada no exemplo reflete decisões de modelagem, organização dos dados e definição de relacionamentos que **impactam diretamente o funcionamento da solução**.
- O script gerado pelo brModelo serve como base, mas deve ser refinado com tipos de dados corretos, restrições **NOT NULL**, **CHECK**, **UNIQUE** e políticas de exclusão em cascata.

Testando com o SSMS

- O **SQL Server Management Studio (SSMS)** é uma ferramenta gráfica gratuita da Microsoft utilizada para gerenciar instâncias do SQL Server.
- É adotada por DBAs, desenvolvedores e analistas de dados para administração, desenvolvimento de consultas e manutenção de bancos de dados.
- **O Que é Possível Fazer com o SSMS**
 - **Estruturas de BD**
Criar, alterar e excluir bancos de dados, tabelas, views e procedimentos armazenados.
 - **Consultas SQL**
Executar scripts SQL e consultar dados de maneira interativa.
 - **Desempenho**
Monitorar o desempenho do servidor e analisar planos de execução.
 - **Segurança**
Gerenciar logins, permissões e políticas de segurança.
 - **Backup e Restauração**
Realizar backups, restaurar bancos e automatizar tarefas via Agente SQL Server.

Interatividade

Analise as afirmações:

- I. A entidade ItemPedido atua como entidade associativa, sendo responsável por resolver o relacionamento muitos-para-muitos entre Pedido e Produto, além de armazenar atributos próprios, como quantidade e subtotal.
- II. No modelo lógico, a definição explícita de chaves primárias (PK) e chaves estrangeiras (FK) é fundamental para garantir a integridade referencial entre as tabelas do sistema.
- III. O modelo físico pode ser executado diretamente a partir do script inicial gerado pelo brModelo, sem necessidade de ajustes, pois já contempla tipos de dados adequados e restrições de integridade.
- IV. O uso do SQL Server Management Studio (SSMS) limita-se exclusivamente à criação de tabelas, não oferecendo recursos para execução de consultas, análise de desempenho ou gerenciamento de segurança.

Interatividade

Está correto o que se afirma em:

- a) I e IV, apenas.
- b) I e II, apenas.
- c) II e III, apenas.
- d) III e IV, apenas.
- e) I e III, apenas.

Resposta

Analise as afirmações:

- I. A entidade ItemPedido atua como entidade associativa, sendo responsável por resolver o relacionamento muitos-para-muitos entre Pedido e Produto, além de armazenar atributos próprios, como quantidade e subtotal.
- II. No modelo lógico, a definição explícita de chaves primárias (PK) e chaves estrangeiras (FK) é fundamental para garantir a integridade referencial entre as tabelas do sistema.
- III. O modelo físico pode ser executado diretamente a partir do script inicial gerado pelo brModelo, sem necessidade de ajustes, pois já contempla tipos de dados adequados e restrições de integridade.
- IV. O uso do SQL Server Management Studio (SSMS) limita-se exclusivamente à criação de tabelas, não oferecendo recursos para execução de consultas, análise de desempenho ou gerenciamento de segurança.

Está correto o que se afirma em:

b) I e II, apenas.

Consultas Complexas no SQL Server

- Como transformar dados operacionais de um sistema de pedido de vendas em informações consolidadas, rankings, percentuais e séries históricas — usando **CTEs**, **funções analíticas** e **funções definidas pelo usuário**.

Por que Consultas Avançadas?

Além do Registro Básico

- Um banco de dados de vendas não serve apenas para registrar clientes, pedidos e pagamentos — ele deve produzir informações que auxiliem gestores a acompanhar resultados, identificar padrões e tomar decisões estratégicas.

O Que é Necessário

- Consolidar dados de várias tabelas.
Realizar cálculos automáticos.
Aplicar regras de negócio diretamente no SGBD.
Gerar rankings, percentuais e séries históricas.

Situação Financeira dos Pedidos

- A empresa precisa saber o valor total vendido, quanto já foi pago e se existe saldo pendente. A solução utiliza **CTEs (Common Table Expressions)** com a cláusula **WITH** para dividir o problema em partes menores.

CTE 1 — TotaisPedido

- Soma os valores da tabela **ItemPedido**, produzindo o total de cada venda agrupado por pedido.

CTE 2 — TotaisPagamento

- Agrupa os registros da tabela **Pagamento** para obter quanto já foi quitado por pedido.

Combinação Final

- Os resultados intermediários são unidos aos dados do cliente para gerar o relatório completo de situação financeira.

Código: Situação Financeira dos Pedidos

```
WITH TotaisPedido AS (  
    SELECT p.id_pedido,  
           p.id_cliente,  
           p.data_pedido,  
           SUM(ip.preco_total) AS total_pedido  
    FROM Pedido p  
    INNER JOIN ItemPedido ip ON ip.id_pedido = p.id_pedido  
    GROUP BY p.id_pedido, p.id_cliente, p.data_pedido  
)  
TotaisPagamento AS (  
    SELECT pg.id_pedido,  
           SUM(pg.valor_pago) AS total_pago  
    FROM Pagamento pg  
    GROUP BY pg.id_pedido  
)
```

Código: Situação Financeira dos Pedidos

```
SELECT tp.id_pedido,
       tp.data_pedido,
       c.nome AS nome_cliente,
       tp.total_pedido,
       COALESCE(tpg.total_pago, 0) AS total_pago,
       tp.total_pedido - COALESCE(tpg.total_pago, 0) AS saldo,
       CASE
           WHEN COALESCE(tpg.total_pago, 0) = 0 THEN 'NAO PAGO'
           WHEN COALESCE(tpg.total_pago, 0) < tp.total_pedido THEN 'PARCIAL'
           ELSE 'PAGO'
       END AS status
FROM TotaisPedido tp
INNER JOIN Cliente c ON c.id_cliente = tp.id_cliente
LEFT JOIN TotaisPagamento tpg ON tpg.id_pedido = tp.id_pedido;
```

Recursos Técnicos da Consulta

LEFT JOIN

- Garante que pedidos **sem pagamento** ainda apareçam no resultado final.

COALESCE

- Substitui valores **nulos** por zero, evitando erros de cálculo no saldo.

CASE

- Transforma a regra de negócio em uma **classificação textual**:
NAO PAGO, PARCIAL ou PAGO.
- A utilização de **CTEs** melhora a legibilidade de consultas longas, pois divide o problema em partes menores e mais fáceis de compreender.

Ranking de Clientes por Faturamento

O Objetivo

- Identificar os clientes que mais faturam, ordenando-os pelo total gasto. Isso facilita decisões comerciais e programas de fidelização.

A Estratégia

- Uma CTE soma os valores de todos os pedidos por cliente. Sobre esse resultado, aplica-se a função analítica **DENSE_RANK()** para gerar o ranking automaticamente.
- **Por que DENSE_RANK()?**
 - DENSE_RANK() pertence ao conjunto das window functions e trabalha sobre um conjunto de linhas definido pela cláusula OVER, criando classificações sem eliminar registros do resultado.

Código: Ranking de Clientes

```
;WITH FaturamentoCliente AS (  
    SELECT p.id_cliente,  
           SUM(ip.preco_total) AS faturamento  
    FROM Pedido p  
    INNER JOIN ItemPedido ip ON ip.id_pedido = p.id_pedido  
    GROUP BY p.id_cliente  
)  
SELECT c.nome,  
       fc.faturamento,  
       DENSE_RANK() OVER (ORDER BY fc.faturamento DESC) AS posicao  
FROM FaturamentoCliente fc  
INNER JOIN Cliente c ON c.id_cliente = fc.id_cliente;
```

Window Functions: Conceito-Chave

- Funções analíticas calculam valores sobre grupos de linhas relacionados, **sem a necessidade de usar GROUP BY** na consulta principal. Isso permite manter o nível de detalhe dos registros enquanto se realizam agregações paralelas.

DENSE_RANK()

- Gera posições sem lacunas no ranking, mesmo com empates.

SUM() OVER()

- Calcula totais acumulados ou por partição sem agrupar linhas.

PARTITION BY

- Reinicia o cálculo para cada grupo definido, como um pedido.

Percentual de Cada Item no Pedido

- Em análises detalhadas, é importante compreender como cada item contribui para o valor total do pedido — mantendo o detalhe por produto, mas calculando o total da venda e o percentual de cada linha simultaneamente.

```
SELECT ip.id_pedido,  
pr.nome_produto,  
ip.preco_total,  
SUM(ip.preco_total) OVER (PARTITION BY ip.id_pedido) AS total_pedido,  
CAST(  
100.0 * ip.preco_total /  
NULLIF(SUM(ip.preco_total) OVER (PARTITION BY ip.id_pedido), 0)  
AS DECIMAL(10,2)  
) AS percentual  
FROM ItemPedido ip  
INNER JOIN Produto pr ON pr.id_produto = ip.id_produto;
```

Detalhes Técnicos: Percentual por Item

PARTITION BY id_pedido

- Reinicia o cálculo do **SUM()** para cada pedido, permitindo o cálculo do total sem perder o detalhe de cada linha.

NULLIF

- Evita **divisão por zero**, retornando NULL em vez de erro.

CAST ... DECIMAL(10,2)

- Controla o **formato do valor percentual**, garantindo duas casas decimais.

Faturamento Mensal e Acumulado

Visão Gerencial

- Observar a evolução das vendas ao longo do tempo: faturamento mensal e total acumulado.

Solução Técnica

- A data do pedido é convertida em referência mensal com **DATEFROMPARTS**. A soma acumulada é gerada com **SUM() OVER(ORDER BY mes_ref)**.

```
;WITH FaturamentoMensal AS (  
SELECT DATEFROMPARTS(YEAR(p.data_pedido), MONTH(p.data_pedido), 1) AS  
mes_ref, SUM(ip.preco_total) AS faturamento_mes  
FROM Pedido p  
INNER JOIN ItemPedido ip ON ip.id_pedido = p.id_pedido  
GROUP BY DATEFROMPARTS(YEAR(p.data_pedido),  
MONTH(p.data_pedido), 1)  
)  
SELECT mes_ref, faturamento_mes, SUM(faturamento_mes) OVER (ORDER BY mes_ref)  
AS acumulado  
FROM FaturamentoMensal  
ORDER BY mes_ref;
```

Funções Definidas pelo Usuário (UDFs)

- O SQL Server permite encapsular regras de negócio em funções reutilizáveis.

Função Escalar

- Recebe o identificador do pedido e devolve o somatório de seus itens como um único valor decimal.

Vantagem Principal

- Centralizam regras de cálculo e reduzem repetição de código.

```
CREATE OR ALTER FUNCTION
dbo.fn_total_pedido
(@id_pedido INT)
RETURNS DECIMAL(18,2)
AS
BEGIN
DECLARE @total DECIMAL(18,2);

SELECT @total =
COALESCE(SUM(preco_total), 0)
FROM ItemPedido
WHERE id_pedido = @id_pedido;

RETURN @total;
END;
GO
```

Função Tabelaada: fn_itens_pedido

Função tabelaada retorna um conjunto de registros, funcionando como uma **view parametrizada**.

- ```
CREATE OR ALTER FUNCTION dbo.fn_itens_pedido
(@id_pedido INT)
RETURNS TABLE
AS
RETURN (
SELECT ip.id_item,
pr.nome_produto,
ip.quantidade,
ip.preco_total
FROM ItemPedido ip
INNER JOIN Produto pr ON pr.id_produto = ip.id_produto
WHERE ip.id_pedido = @id_pedido
);
GO
```

# Interatividade

Analise as afirmações:

- I. As CTEs (Common Table Expressions) permitem dividir consultas complexas em blocos lógicos, melhorando a legibilidade e a manutenção do código, sem a necessidade de criar objetos permanentes no banco de dados.
- II. Funções analíticas como `SUM() OVER()` e `DENSE_RANK()` exigem obrigatoriamente o uso de `GROUP BY` na consulta principal, pois operam apenas sobre dados previamente agregados.
- III. O uso de `PARTITION BY` em funções analíticas possibilita realizar cálculos por grupo, mantendo o nível de detalhe das linhas originais do resultado.
- IV. Funções definidas pelo usuário do tipo tabelada podem retornar múltiplos registros e ser utilizadas como uma view parametrizada em consultas SQL.

# Interatividade

Está correto o que se afirma em:

- a) I e II, apenas.
- b) II e III, apenas.
- c) I e IV, apenas.
- d) I e III, apenas.
- e) II e IV, apenas.

# Resposta

Analise as afirmações:

- I. As CTEs (Common Table Expressions) permitem dividir consultas complexas em blocos lógicos, melhorando a legibilidade e a manutenção do código, sem a necessidade de criar objetos permanentes no banco de dados.
- II. Funções analíticas como SUM() OVER() e DENSE\_RANK() exigem obrigatoriamente o uso de GROUP BY na consulta principal, pois operam apenas sobre dados previamente agregados.
- III. O uso de PARTITION BY em funções analíticas possibilita realizar cálculos por grupo, mantendo o nível de detalhe das linhas originais do resultado.
- IV. Funções definidas pelo usuário do tipo tabelada podem retornar múltiplos registros e ser utilizadas como uma view parametrizada em consultas SQL.

Está correto o que se afirma em:

d) I e III, apenas.

# Referências

- ALVES, William P. *Banco de dados: teoria e desenvolvimento*. Rio de Janeiro: Érica, 2021. *E-book*.
- DATE, C. J. *Introdução a sistemas de bancos de dados*. Rio de Janeiro: LTC, 2023.
- ELMASRI, R.; NAVATHE, S. B. *Sistemas de banco de dados*. 7. ed. São Paulo: Pearson, 2018. *E-book*.
- HEUSER, Carlos Alberto. *Projeto de bancos de dados*. Porto Alegre: Bookman, 2009.
- MACHADO, Felipe Nery R. *Banco de dados – projeto e implementação*. Rio de Janeiro: Érica, 2020. *E-book*.
- PICHETTI, Roni F.; VIDA, Edinilson S.; CORTES, Vanessa S. M. P. *Banco de dados*. Porto Alegre: SAGAH, 2021. *E-book*.
- SILBERSCHATZ, Abraham. *Sistema de banco de dados*. Rio de Janeiro: GEN LTC, 2020. *E-book*.
- SILVA, Luiz F. C.; RIVA, Aline D.; ROSA, Gabriel A. *et al.* *Banco de dados não relacional*. Porto Alegre: SAGAH, 2021. *E-book*.

**ATÉ A PRÓXIMA!**